

Production Craft — Persistence, Approval, Durability, Multi-Agent

Track 16.4 —  LangGraph · 16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md

Verified against **LangGraph 1.2.x** / `langgraph-checkpoint 4.2.x`, September 2026. Sources: [persistence](#), [interrupts](#), [streaming](#), [multi-agent](#).

The first note was about *building* a graph. This one is about the four things that decide whether anyone lets you run it: it remembers, it can be stopped, it survives a crash, and you can see inside it.

1. Persistence: one argument, an entirely different system

```
graph = builder.compile(checkpointer=saver)
graph.invoke(inputs, config={"configurable": {"thread_id": "ticket-4471"}})
```

That is the whole API. What you get for it:

- **Memory between turns.** Second `invoke` on the same `thread_id` starts from the stored state, so a chat agent's history is not your problem any more.
- **Resume after a crash or deploy.** The thread is in Postgres, not in a process.
- **Interrupts.** No `checkpointer`, no `interrupt()` — there is nowhere to pause *to*.
- **Time travel.** `get_state_history()` gives you every superstep, newest first.
- **An audit trail** you did not have to write. Every state transition, timestamped, replayable.

Threads and checkpoints

A **thread** is a lineage (one conversation, one case, one run). A **checkpoint** is one immutable snapshot inside it, addressed by `checkpoint_id`. `get_state(cfg)` returns the *newest* checkpoint on the thread; `get_state({"configurable": {"thread_id": t, "checkpoint_id": c}})` returns a specific one. A `StateSnapshot` carries `values`, `next` (the nodes queued to run), `config`, `metadata`, `tasks` and `interrupts`.

Choosing a checkpointer

Checkpointer	Package	Use for
InMemorySaver	langgraph-checkpoint (bundled)	tests, notebooks, demos. Dies with the process.
SqliteSaver / AsyncSqliteSaver	langgraph-checkpoint- sqlite	single-node apps, desktop/CLI agents, local dev
PostgresSaver / AsyncPostgresSaver	langgraph-checkpoint- postgres	anything with >1 replica or an SLA. Call .setup() once.

MemorySaver is still importable as an alias of InMemorySaver; new code should say InMemorySaver. Two operational facts nobody mentions until it hurts:

- **Checkpoints accumulate.** One write per superstep per thread, forever, unless you delete them. Budget for it and write a retention job — which, conveniently, is also what your data-retention policy requires (../.../14-ai-compliance/).
- **Keep thread_id short** (under 255 chars) and opaque. Hash natural keys; do not concatenate a user email and a timestamp into it.

Checkpoints vs. Store

A checkpointer is **thread-scoped short-term memory**. A Store (from langgraph.store.memory import InMemoryStore, or the Postgres store in production) is **cross-thread long-term memory**: namespaced key-value data that survives beyond one conversation — user preferences, learned facts, per-tenant settings. Nodes reach it via Runtime.store. Do not abuse state for this; a 300-turn thread should not be carrying a user's timezone in every snapshot.

2. Human-in-the-loop: the feature that makes agents deployable

```
from langgraph.types import interrupt, Command

def approval_gate(state):
    decision = interrupt({"question": "Approve this refund?",
                        "amount": state["proposed"]})    # graph stops here
    return {"decision": decision}

# ... later, possibly in a different process, on a different day:
graph.invoke(Command(resume="approve"), config={"configurable": {"thread_id": t}})
```

The key property: **an interrupted run returns**. It does not block a worker, hold a socket, or keep a process alive. invoke() comes back with an `__interrupt__` entry containing your payload, and `get_state(cfg).next` tells you which node is waiting. Your web app writes that to a queue, sends a Slack message, and forgets about it. Approval three hours later costs you nothing in the meantime.

Four patterns, all the same primitive:

Pattern	Resume value	What the node does
Approve / reject	"approve" / "reject"	route to execute or cancel
Edit the proposed action	{"action": "edit", "amount": 175}	write the human's value into state
Review a tool call	the corrected args	call <code>interrupt()</code> <i>inside the tool</i> , before the side effect
Ask a clarifying question	free text	append it as a <code>HumanMessage</code> and loop back

The re-execution trap

Resuming re-runs the node from the top. LangGraph does not freeze a Python stack frame; it replays the node and substitutes your resume value where `interrupt()` raised. So this is a bug:

```
def approval_gate(state):
    send_email(...)          # <- fires AGAIN on resume
    decision = interrupt(...)
```

Run `code/checkpoint_and_interrupt.py` [16-ai-tech-stack/tracks/langgraph/code/checkpoint_and_interrupt.py] section 4 and watch the notification fire twice per gate. Fixes, in order of preference: put the side effect in its own node; make it idempotent with a key derived from `(thread_id, checkpoint_id, request_id)`; or use `@task` from the functional API so the completed result is memoised against the checkpoint.

The same rule governs crash recovery — a node that died halfway is re-run *whole*. **Design every node so that running it twice is boring.** That is the single most important sentence in this note.

Static breakpoints

`compile(interrupt_before=["tools"]) / interrupt_after=[...]` pause without a code change. They are a *debugging* tool — great in LangGraph Studio, wrong for a product feature, because they carry no payload for the human to look at. Use `interrupt()` for anything a user sees.

Why governance cares

An approval gate turns "the model did something" into "a named person authorised this at 14:32, and here is the exact state they saw". That is the difference between a demo and a system that ships in a regulated environment. Cross-reference `../12-ai-governance/` for human-oversight obligations and `../13-ai-security/` for why the gate is also a defence against prompt-injected tool calls.

3. Durable execution, retries, and idempotency

Durability modes — pass `durability=` to `invoke/stream`:

Mode	When checkpoints are written	Trade
"sync"	before the next step starts (default)	safest; one blocking write per superstep
"async"	in parallel with the next step	faster; a crash can lose the last step
"exit"	once, at the end of the run	fastest; no mid-run recovery, no interrupts

Retries — per node, for *transient* failures:

```
from langgraph.types import RetryPolicy
builder.add_node("fetch", fetch, retry_policy=RetryPolicy(max_attempts=3, initial_interval=0.5,
                                                         backoff_factor=2.0, jitter=True))
```

`add_node` also takes `timeout=`, `cache_policy=CachePolicy(ttl=...)` for memoising expensive deterministic nodes, and `error_handler=` for routing a failure somewhere useful.

Know the difference, because people reach for the wrong one constantly:

- **Retry** fixes *flaky infrastructure*: a 502, a timeout, a rate limit. Same inputs, try again.
- **Returning the error to the model** fixes *wrong arguments*: a bad order id, a malformed query. Retrying identical bad arguments three times is just a slower failure. Return `ToolMessage(status="error")` and let the model correct itself — see `code/react_agent_from_scratch.py` [16-ai-tech-stack/tracks/langgraph/code/react_agent_from_scratch.py] section 2.

Use both. They solve different problems.

Determinism. Because resume replays nodes, a node that reads `datetime.now()` or `random()` above an interrupt can produce a *different* result on replay, and your state quietly diverges. Capture non-deterministic values into state once, in their own node, and read them from state afterwards.

4. Streaming: what you show a human while it thinks

An agent that takes 40 seconds and shows nothing feels broken. Five modes, and you will use three:

stream_mode	Emits	Use for
"updates"	{node: partial_update} after each node	the default for a UI — it's the diff, cheap and high-signal
"values"	the full state after each step	replacing a rendered view wholesale; verbose
"messages"	(token, metadata) from LLM calls	token-by-token typing effect
"custom"	whatever you write via <code>get_stream_writer()</code>	"searching 12 documents...", progress bars, citations
"debug" / "tasks" / "checkpoints"	execution internals	diagnosis, not product

Combine them — `stream_mode=["updates", "custom"]` yields `(mode, payload)` tuples — and pass `subgraphs=True` to see inside nested graphs (each chunk then carries its namespace).

"custom" is the underrated one. Your node knows things the framework cannot infer:

```
from langgraph.config import get_stream_writer

def research(state):
    writer = get_stream_writer()
    writer({"status": "searching", "n": len(queries)})
    ...
```

That is how you turn a black box into a progress log without polluting state. A practical UI recipe: "custom" for status lines, "messages" for the answer text, "updates" to drive a step indicator.

5. Multi-agent: patterns, and when one agent wins

The primitives are unremarkable — that is the point. A "multi-agent system" in LangGraph is subgraphs plus `Command(goto=..., graph=Command.PARENT)`.

SUPERVISOR [sup] / \ [a] [b] [c] all routing via sup easy to trace/limit sup is the bottleneck	HAND-OFF (swarm) [a] <-> [b] \ / [c] peer-to-peer fewer hops, harder to reason about	HIERARCHICAL [top] / \ [team1] [team2] sup of sups scales org-shaped problems, lots of latency
---	---	---

Pattern	Shape	Good at	Fails at
Supervisor	one router, N workers as tools/subgraphs	traceability, per-worker limits, easy eval	supervisor context bloat; every hop is 2 extra model calls
Hand-off / swarm	agents transfer control to each other	fewer hops, natural for "escalate to billing"	control flow is emergent; loops between two agents are real
Hierarchical teams	supervisors of supervisors	genuinely large task trees	latency and cost multiply; debugging is a job
Router	classify once, then a specialist	cheap, predictable	no recovery if the classification was wrong

The honest advice: most "multi-agent" systems should be one agent with better tools. Each extra agent buys you context isolation and costs you a round trip, a serialization boundary, and a place for information to be lost in a hand-off summary. LangChain's own guidance says the quiet part out loud: *not every complex task requires this approach — a single agent with the right tools and prompt can often achieve similar results.*

Use multiple agents when at least one is true:

- One agent's context genuinely does not fit, or the tool list is so long the model mis-selects.

- Subtasks are **independent** and can run in parallel (research fan-out is the canonical win).
- Different subtasks need different models, permissions, or data residency.
- You need per-role approval gates — "the refund agent requires a human, the lookup agent does not".

Do **not** split because the org chart has three teams. That produces agents that spend their tokens explaining themselves to each other. Anthropic's multi-agent research write-up measured roughly 15× the token spend of a single chat for their fan-out research system — worth it there, absurd for a support bot. Read it alongside Cognition's "Don't Build Multi-Agents" (both linked in `../papers/PAPERS.md`) and form your own view; the disagreement is real and the answer is task-shaped.

Prebuilt libraries exist (`langgraph-supervisor`, `langgraph-swarm`) and are fine for a first pass, but they are thin wrappers — read their source before adopting, it is a short afternoon.

See `../../07-agentic-ai/` for the topology theory this implements.

6. Observability and debugging

Graphs fail in ways loops do not: the wrong edge fired, a channel got clobbered, a node silently returned `{}`, the loop ran 20 times. Your toolkit, in the order you should reach for it:

1. `stream_mode="updates"` — 90% of bugs are visible in the per-node diff. Free, no setup.
2. `get_state_history()` — for anything already finished: what did state look like at step 4?
3. `graph.get_graph().draw_mermaid()` — print the topology and check it matches your intent. (`draw_ascii()` needs `pip install grandalf`.)
4. **LangGraph Studio** via `langgraph dev` — visual run inspection, breakpoints, editing state mid-run. Best debugger in the ecosystem; see `../langsmith/`.
5. **LangSmith tracing** — every node becomes a span with inputs, outputs, latency, tokens and cost, automatically, once `LANGSMITH_TRACING=true` is set. This is where "which node burns the budget" gets answered.

Failure modes to recognise on sight:

Symptom	Almost always
<code>InvalidUpdateError: Can receive only one value per step</code>	two parallel nodes wrote a channel with no reducer
A list keeps getting shorter	missing <code>Annotated[..., operator.add]</code> — silent overwrite
<code>GraphRecursionError</code>	no budget in state; the model never emits a stopping condition
Provider rejects the next call	a tool call without a matching <code>ToolMessage</code>
A side effect happened twice	code above <code>interrupt()</code> , or a replayed node
"It worked locally"	<code>InMemorySaver</code> in prod, or <code>durability="exit"</code>

7. Deployment notes

You do not need LangGraph Platform. A compiled graph is a Python object: put it behind FastAPI, give it a `PostgresSaver`, and you have a service. Do that first if your infra team already has opinions.

What the managed **Agent Server** (the LangGraph Platform runtime, launched locally with `langgraph dev`, packaged via `langgraph build/langgraph up`, configured by `langgraph.json`) adds, and what you would otherwise build yourself:

- **Assistants / threads / runs / cron** as first-class REST resources.
- A **task queue** (Redis for signalling and streaming pub/sub, Postgres for run data) so long runs are not tied to an HTTP request.
- **Streaming and cancellation** over the wire, including reconnect.
- **Double-texting policies** — what happens when a user sends a second message while a run is in flight (interrupt it, enqueue it, reject it). You will need an answer to this; it is easy to underestimate.
- Deployment shapes ranging from fully managed cloud to self-hosted in your own VPC.

Rules of thumb: prototype with `langgraph dev`; ship the graph inside your own service if runs are short and you already have Postgres; adopt the Server when you need queued long-running runs, scheduled agents, or multi-tenant thread management and would rather not write that twice.

Whatever you choose: `PostgresSaver`, a `thread_id` you can trace back to a user, a retention job, and tracing on from day one. Those four decisions are hard to retrofit and boring to do up front.